arXiv:2309.00908v1 [cs.CV] 2 Sep 2023

MagicProp: Diffusion-based Video Editing via Motion-aware Appearance Propagation

Hanshu Yan,* **Jun Hao Liew,*** **Long Mai,** **Shanchuan Lin &** **Jiashi Feng**
ByteDance Inc.
{hanshu.yan, junhao.liew, long.mai, peterlin, jshfeng}@bytedance.com

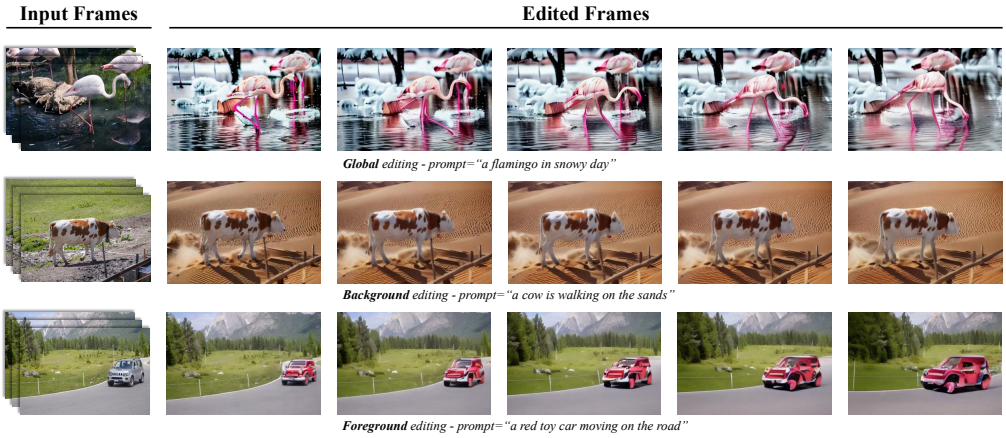

Figure 1: Video editing via MagicProp: global, background, and foreground editing are all supported.

Abstract

This paper addresses the issue of modifying the visual appearance of videos while preserving their motion. A novel framework, named MagicProp, is proposed, which disentangles the video editing process into two stages: appearance editing and motion-aware appearance propagation. In the first stage, MagicProp selects a single frame from the input video and applies image-editing techniques to modify the content and/or style of the frame. The flexibility of these techniques enables the editing of arbitrary regions within the frame. In the second stage, MagicProp employs the edited frame as an appearance reference and generates the remaining frames using an autoregressive rendering approach. To achieve this, a diffusion-based conditional generation model, called PropDPM, is developed, which synthesizes the target frame by conditioning on the reference appearance, the target motion, and its previous appearance. The autoregressive editing approach ensures temporal consistency in the resulting videos. Overall, MagicProp combines the flexibility of image-editing techniques with the superior temporal consistency of autoregressive modeling, enabling flexible editing of object types and aesthetic styles in arbitrary regions of input videos while maintaining good temporal consistency across frames. Extensive experiments in various video editing scenarios demonstrate the effectiveness of MagicProp.

1 Introduction

Content creation often involves video editing, which includes modifying the appearance or adjusting the motion of raw videos [Wu et al., 2023; Kasten et al., 2021; Zhao et al., 2023; Wang et al., 2023]. Filmmakers may need to adjust the exposure, saturation, and contrast of raw videos for better aesthetic

*Equal contribution.

quality, while advertisers may want to change realistic videos into certain fascinating styles to impress target audiences. This paper addresses the problem of editing videos' appearance, including changing the content or style locally in a certain spatial region or globally throughout the entire video.

Existing works attempt to solve this problem mainly from two perspectives: editing each frame individually via image generation models [Qi et al., 2023; Ceylan et al., 2023; Yang et al., 2023; Khachatryan et al., 2023; Geyer et al., 2023] or modeling the entire video sequence for appearance changing [Ni et al., 2023; Molad et al., 2023; Karras et al., 2023; Kasten et al., 2021; Esser et al., 2023]. Methods based on image models, such as Stable Diffusion [Rombach et al., 2022] and ControlNet [Zhang and Agrawala, 2023], can flexibly modify the content or style of any arbitrary region, but it is challenging to ensure temporal consistency across adjacent frames. To alleviate this issue, some use structure-guided models and cross-frame attention to align color and layout across frames [Zhang and Agrawala, 2023; Qi et al., 2023; Ceylan et al., 2023]. Other methods exploit inter-frame correspondence, such as optical flow, to warp the features of edited frames [Yang et al., 2023; Geyer et al., 2023]. However, the temporal consistency of the edited video is still suboptimal. Instead of using image-based models, researchers have developed many sequence-based models for video generation and editing [Esser et al., 2023; Couairon et al., 2023]. Neural Layered Atlas (NLA) overfits a video first and then edits the learned corresponding Atlas to change the foreground or background [Kasten et al., 2021; Bar-Tal et al., 2022]. NLA-based methods can effectively edit the appearance of videos, but test-time optimization is time- and resource-consuming. Recently, many diffusion-based models have been proposed for structure-aware video generation, such as Gen-1 [Esser et al., 2023], ControlVideo [Zhao et al., 2023; Chen et al., 2023], and VideoComposer [Wang et al., 2023]. These methods synthesize videos by conditioning on layout sequences such as depth or sketch maps, so that the motion coherence in the resultant video can be ensured. However, the editability and flexibility will be compromised due to the limitation of textual descriptions and the difficulty of user interaction. For instance, when editing a certain part of a given video, text prompts may not precisely localize the region of interest across all frames, and it may be challenging for users to prepare masks for all frames. The trade-off between temporal consistency and editing flexibility inspires us to explore other alternative frameworks for video editing.

Motivated by the fact that frames within a video usually share a similar scene, we propose a novel framework, MagicProp, which disentangles video editing into two stages, namely, appearance editing and motion-aware appearance propagation. MagicProp first selects one frame from the given video and edits its appearance. The edited frame is used as the appearance reference in the second stage. Then, MagicProp autoregressively renders the remaining frames by conditioning on the reference frame and the motion sequence (*e.g.*, depth maps of the given video). MagicProp models videos in an autoregressive manner, which guarantees the temporal consistency of the output videos. Additionally, MagicProp uses powerful image diffusion models (optionally with additional masks) for reference editing, allowing for flexible modification of the contents of a local region or the entire video.

The most crucial component of MagicProp is an autoregressive conditional image diffusion model that synthesizes the target image under the control of its previous frame, the target depth, and the reference appearance. We design a lightweight adapter to merge and inject the semantic-level and pixel-level information of the reference frame into the image generation process, ensuring that the appearance of the resultant frames aligns well with the reference. During training, we follow the strategy of zero terminal signal-to-noise ratio (SNR) [Lin et al., 2023], which bridges the gap between the noise schedules during training and inference, resulting in better matching of the color and style of generated frames with the reference. We conducted extensive experiments in several video editing scenarios, including local object/background editing and global stylization. The results demonstrate the effectiveness and flexibility of MagicProp. The contributions of MagicProp are three-fold:

- We proposed a novel framework, MagicProp, that decouples video editing into appearance editing and motion-aware appearance propagation.
- We devised a lightweight adapter to inject class- and pixel-level features into the diffusion model. We also applied the zero-terminal SNR strategy for training. These techniques facilitate the alignment of the appearance.
- Extensive experiments demonstrate that MagicProp can flexibly edit any arbitrary region of the given video and generate high-quality results.

2 Related Works and Preliminaries

In this section, we first review recent related works on the appearance editing of videos. We categorize them into two groups, *i.e.*, editing a video frame by frame via image models, and modeling the whole frame sequence for editing. Then, we introduce the preliminaries about diffusion probabilistic models and the notation for video editing.

2.1 Related Works

Frame-by-frame Editing Diffusion-based image generation models have achieved great success in image generation and editing tasks [Ho et al., 2020; 2022; Rombach et al., 2022; Blattmann et al., 2023]. The simplest method for video editing is to edit each frame individually [Meng et al., 2022; Liew et al., 2022; Hertz et al., 2022]. Although it is flexible to edit each frame and the resultant frames have a good aesthetic quality, the temporal consistency of the whole video is usually inferior. Some methods use the layout condition generation method to edit each frame [Zhang and Agrawala, 2023; Huang et al., 2023b]. For example, ControlNet [Zhang and Agrawala, 2023] synthesizes images with the conditioning of a text description and an additional layout map, such as a depth map or an edge map, thus the spatial layout of the edited frame matches that of the original frame. Whilst these methods can guarantee the layout consistency of the edited videos, the appearance of frames (*e.g.*, identity, texture, and color) still changes apparently across frames. To alleviate the issue of temporal consistency, a line of methods rely on cross-frame attention to fuse the latents of edited frames and those of their previous frames (or other reference frames) [Qi et al., 2023; Hertz et al., 2022; Khachatryan et al., 2023; Ceylan et al., 2023], so that the consistency of shape and style can be improved. Another line of methods exploit the correspondence between frames in the original video and use it to warp the latent or attention maps when generating future frames [Yang et al., 2023; Geyer et al., 2023]. Correspondence-based wrapping may fail due to the occlusion in consecutive frames. In general, methods based on per-frame editing still suffer from temporal consistency across frames.

Editing via Sequential Modeling Videos are naturally sequential data, and therefore using sequential models for video generation and editing intrinsically benefits temporal consistency. Neural Layered Atlas (NLA) [Kasten et al., 2021; Bar-Tal et al., 2022; Huang et al., 2023a] represents a video through several 2D maps and 2D-to-color atlases. The appearance of objects and backgrounds can be easily edited by modifying the corresponding atlases. However, NLA needs to perform test-time optimization for each video to learn its representations, which is very time-consuming. Recently, diffusion models have been proven effective in modeling sequential data like videos. Many methods use video diffusion models or flatten image diffusion models into video models for video editing [Ho et al., 2022; Blattmann et al., 2023; Zhou et al., 2023; Wang et al., 2023]. Dreamix [Molad et al., 2023] and Tune-A-Video [Wu et al., 2023], fine-tune the video model on the provided video first and then generate a new video by conditioning the textual prompt of the editing instruction. Fine-tuning on the given video cannot sufficiently guarantee that the motion (layout sequence) in the edited video aligns well with the original. To ameliorate this issue, motion-conditioned video diffusion models have been proposed, including Gen-1 [Esser et al., 2023], ControlVideo [Zhao et al., 2023; Chen et al., 2023], and VideoComposer [Wang et al., 2023]. These methods generate video with the condition of a layout sequence, such as depth or edge maps. When editing, one can extract the layout sequence from the given video first and then generate a new video by conditioning the layout sequence and an editing text prompt. Overall, editing methods based on video models can effectively synthesize temporally consistent videos, but their editability and image quality are not as good as the image-based models at the current stage due to the limitation of textual description and the difficulty of training a good video model. Textual prompts only can provide a high-level semantic description of the desired appearance. It is challenging to locate a specific local editing region of a video based on textual prompts.

In contrast, MagicProp disentangles appearance editing and appearance propagation. It can flexibly edit the appearance based on powerful image editing methods that can incorporate textural descriptions and localization masks. Besides, synthesizing future frames with an autoregressive model also ensures temporal consistency across frames.

2.2 PRELIMINARIES

Denoising Diffusion Probabilistic Model Denoising diffusion probabilistic models (DDPM) are a family of latent generative models that approximate the probability density of training data by reversing the Markovian Gaussian diffusion processes [Sohl-Dickstein et al., 2015; Ho et al., 2020]. Concerning a distribution $q(\boldsymbol{x})$, DDPM models the probability density $q(\boldsymbol{x})$ as the marginal of the joint distribution between $\boldsymbol{x}$ and a series of latent variables $x_{1:T}$, *i.e.*, $p_\theta(\boldsymbol{x}) = \int p_\theta(\boldsymbol{x}_{0:T}) d\boldsymbol{x}_{1:T}$ with $\boldsymbol{x} = \boldsymbol{x}_0$. The joint distribution is defined as a Markov chain with learned Gaussian transitions starting from the standard normal distribution, *i.e.*,

$$p_\theta(\boldsymbol{x}_T) = \mathcal{N}(\boldsymbol{x}_T; \mathbf{0}, \boldsymbol{I}) \tag{1}$$

$$p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{x}_{t-1}; \boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t), \Sigma_\theta(\boldsymbol{x}_t, t)) \tag{2}$$

To perform likelihood maximization of the parameterized marginal $p_\theta(\cdot)$, DDPM uses a fixed Markov Gaussian diffusion process, $q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0)$, to approximate the posterior $p_\theta(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0)$. In specific, two series, $\alpha_{0:T}$ and $\sigma^2_{0:T}$, are defined, where $1 = \alpha_0 > \alpha_1 > \ldots, > \alpha_T \geq 0$ and $0 = \sigma_0^2 < \sigma_1^2 < \cdots < \sigma_T^2$. For any $t > s \geq 0$, $q(\boldsymbol{x}_t|\boldsymbol{x}_s) = \mathcal{N}(\boldsymbol{x}_t; \alpha_{t|s}\boldsymbol{x}_s, \sigma^2_{t|s}\boldsymbol{I})$, where $\alpha_{t|s} = \alpha_t/\alpha_s$ and $\sigma^2_{t|s} = \sigma_t^2 - \alpha^2_{t|s}\sigma_s^2$. Usually, we set $\alpha_t^2 + \sigma_t^2 = 1$, thus,

$$q_(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_t|\alpha_t\boldsymbol{x}_0, (1-\alpha_t^2)\boldsymbol{I}). \tag{3}$$

We use deep neural networks to parameterize the expectation function $\mu_\theta(\boldsymbol{x}_t, t)$ of the sampling process or the denoising function $\epsilon_\theta(\boldsymbol{x}_t, t)$, which can be used to alternatively estimate the expectation via $\mu_\theta(\boldsymbol{x}_t, t) = \frac{1}{\sqrt{\alpha_{t|t-1}}}(\boldsymbol{x}_t - \frac{1-\alpha_{t|t-1}}{\sqrt{1-\alpha_t}}\epsilon_\theta(\boldsymbol{x}_t, t))$. When performing conditional generation tasks, the network should take additional control signals $\boldsymbol{y}$ as input, *i.e.*, $\epsilon_\theta(\boldsymbol{x}_t, t, \boldsymbol{y})$. The parameterized reversed process p_θ can be optimized by maximizing the associated evidence lower bound (ELBO). We plug the Gaussian parameterization into KL-divergence terms, the ELBO optimization turns to be noise estimation, where $\lambda(t)$ is a weighting function. After training, we can sample new data via the Markov chain defined in Eqn (2). Instead, we also can use deterministic samplers, such as DDIM, to generate new data. For a certain starting noise $\boldsymbol{x}_T \sim \mathcal{N}(\boldsymbol{x}_T; \mathbf{0}, \boldsymbol{I})$, the mapping from $\boldsymbol{x}_T$ to the generated datum $\boldsymbol{x}_0$ through a deterministic sampler is denoted by $\Phi(\boldsymbol{x}_T, \boldsymbol{y})$.

$$L = \mathbb{E}_{\boldsymbol{x}_0, t, \epsilon}[\lambda(t)\|\epsilon_\theta(\boldsymbol{x}_t) - \epsilon\|_2^2]. \tag{4}$$

Notation for Video Editing We denote a video by $\boldsymbol{x} = [\boldsymbol{x}^1, ..., \boldsymbol{x}^K]$, where $\boldsymbol{x}^i$ represents the ith frame in the sequence and, for each $i \in [1, \ldots, K]$, $\boldsymbol{x}^i \in [-1, 1]^{C\times H\times W}$. To reduce the computational overhead of modeling videos, we use a variational auto-encoder (VAE), denoted by $\{\mathcal{E}(\cdot), \mathcal{D}(\cdot)\}$, to map videos from the RGB space to a lower-dimensional latent space. The video frames are transformed one by one, *i.e.*, $\boldsymbol{z} = [\boldsymbol{z}^1, ..., \boldsymbol{z}^K]$ with $\boldsymbol{z}^i = \mathcal{E}(\boldsymbol{x}^i)$. We follow Stable Diffusion which uses an encoder to downsample $\boldsymbol{x}$ into a spatially $8\times$ smaller space. The generated latent codes can be decoded to videos by $\mathcal{D}(\cdot)$. The editing operations require users to provide extra information describing the desired appearance of the target video. We denote the instruction information by $\boldsymbol{y}$; it could be a textual description, an extra localization mask, or other visual reference. We use CLIP, denoted by $\tau(\cdot)$, to encode the text prompt or reference image, and the embedding is denoted $\tau(\boldsymbol{y})$. To preserve the motion of the original video, we use a depth estimation model, such as TCMonoDepth, to extract the sequence of depth maps for representing the motion. We denote $\mathcal{M}(\cdot)$ as the depth model and $\boldsymbol{m} = [\boldsymbol{m}^1, \ldots, \boldsymbol{m}^K]$ with $\boldsymbol{m}^i = \mathcal{M}(\boldsymbol{x}^1)$ as the depth sequence.

3 METHOD

This paper addresses the problem of motion-preserving video editing, where we aim to alter the appearance of a given video while retaining the original motion. Typically, frames in a short video have similar scenes, with main objects and backgrounds appearing consistently throughout. It is natural to disentangle the video editing problem into two sub-tasks, *viz.*, editing the appearance of the main objects and/or the background first and then propagating the edited content to all other frames based on the original motion.

In this section, we elucidate the pipeline of MagicProp $\mathcal{V}(\cdot)$, which performs video editing in two stages sequentially, *i.e.*, appearance editing $\Phi^1(\cdot)$ and motion-aware appearance propagation $\Phi^2(\cdot)$.

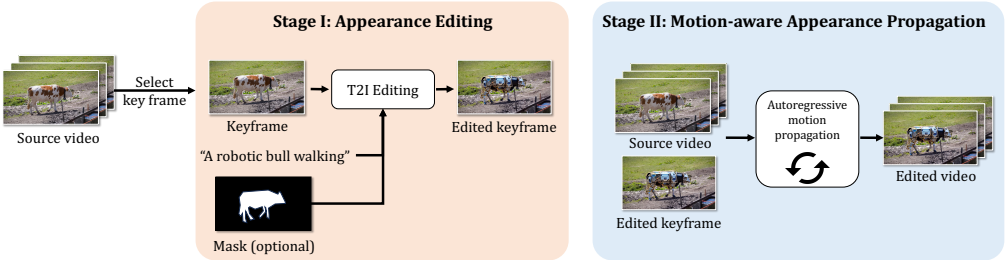

Figure 2: The pipeline of MagicProp.

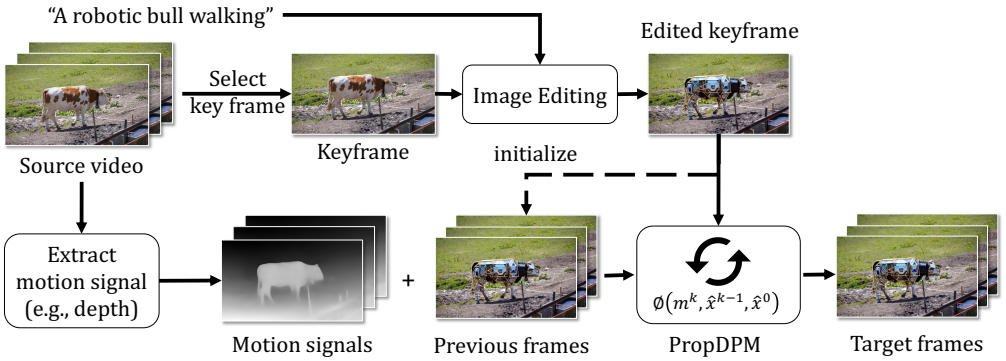

Figure 3: Auto-regressive Motion-aware Appearance Propagation Diffusion Model

MagicProp can flexibly edit the appearance of a given video according to users' instructions. It supports changing the contents (*e.g.*, object type and image style) in any specific region, either locally or globally. Formally, MagicProp takes input as the source video $\boldsymbol{x}$, a textual prompt $\boldsymbol{y}$, and optionally a localization mask $\boldsymbol{w}$. This mask can be provided by users or easily obtained by a powerful segmentation model. After the two-stage processing, MagicProp generates an edited video $\hat{\boldsymbol{x}}$ whose motion remains unchanged.

3.1 Appearance Editing

The first stage of MagicProp is to manipulate the appearance of the source video. We select one frame as the appearance reference. Thanks to many effective image-editing methods, we can flexibly edit any arbitrary region of the reference frame, including changing object types or visual styles.

In specific, we select a frame $\boldsymbol{x}^{\#}$ from the input video $\boldsymbol{x}$ as the appearance reference. Existing image editing methods, such as Text-to-Image (T2I) models, offer rich possibilities to manipulate images' contents [Meng et al., 2022; Liew et al., 2022; Zhang and Agrawala, 2023]. Here, we use the ControlNet optionally with a segmentation mask $\boldsymbol{w}$ to change the main objects and/or the background. By conditioning the depth map of $\boldsymbol{x}^{\#}$ and a textual prompt $\boldsymbol{y}$, ControlNet will generate a new image $\hat{\boldsymbol{x}}^{\#}$ whose layout matches the original one and semantics aligns with the text description. In comparison to existing Text-to-Video (T2V) models, T2I models, such as Stale Diffusion, have apparent superiority in terms of per-frame quality. Thus, the resultant frame edited by ControlNet contains rich details and enjoys high aesthetic quality. Besides, T2I diffusion models allow us to use localization masks to precisely control the editing parts in images. It is flexible to edit a local region or the whole image. In brief, stage one chooses and edits a certain frame, and the edited frame will be used as the appearance reference for video synthesis in the second stage.

$$\hat{\boldsymbol{x}}^{\#} = \Phi^1(\boldsymbol{x}, \#, \boldsymbol{y}, \boldsymbol{w}) \tag{5}$$

3.2 MOTION-AWARE APPEARANCE PROPAGATION

Given a source video $\boldsymbol{x}$ and the appearance reference $\hat{\boldsymbol{x}}^{\#}$, the second stage $\Phi^2(\cdot)$ will render a new video $\hat{\boldsymbol{x}}$ that preserves the motion in source one and whose appearance matches the reference. The most crucial part is an appearance propagation diffusion probabilistic model (PropDPM). PropDPM, denoted by $\phi_\theta(\cdot)$, synthesizes the whole video in an auto-regressive manner. Each frame $\hat{\boldsymbol{x}}^k$ is generated with the conditioning of the reference appearance $\hat{\boldsymbol{x}}^{\#}$, its corresponding depth map $\boldsymbol{m}^k$, and the previous edited frame $\hat{\boldsymbol{x}}^{k-1}$. We can use the edited appearance reference as the starting frame, *i.e.*, $\hat{x}^0 = \hat{x}^{\#}$ and $\boldsymbol{m}^0 = \boldsymbol{m}^{\#}$. The rest can be rendered frame-by-frame through Eqn (6) for k from 1 to K. The layout in the generated frames aligns with the depth maps extracted from the corresponding frames in the source video. Hence, the motion (layout sequence) remains unchanged compared to the source video, and the temporal consistency in the rendered video is also guaranteed.

$$\hat{\boldsymbol{x}}^k = \phi_\theta(\boldsymbol{m}^k, \hat{\boldsymbol{x}}^{k-1}, \boldsymbol{m}^{k-1}, \hat{\boldsymbol{x}}^{\#}) \tag{6}$$

$$\hat{\boldsymbol{x}} = \Phi^2(\hat{x}^{\#}, \boldsymbol{x}) \tag{7}$$

In specific, PropDPM is designed based on the latent diffusion model [Rombach et al., 2022]. We use a VAE $\{\mathcal{E}(\cdot), \mathcal{D}(\cdot)\}$ to map a video into a lower-dimensional latent space. PropDPM is trained to generate the edited latent $\hat{\boldsymbol{z}}^k$ and we then use the VAE to reconstruct the edited video frame $\hat{\boldsymbol{x}}^k$. For the conditioning signals, we split them into two groups, *viz.*, the spatial conditions and the semantic conditions. The spatial conditions, including the target frame's depth map and the previous frame, provide the spatial layout information for the generated image and form a contrast between two consecutive frames. This contrast facilitates the synthesis of contents by querying spatially corresponding regions. The semantic conditions include the RGB and the latent of the reference frame. They provide information about the color, style, and object classes in the target edited video.

The spatial conditions are injected into the PropDPM by concatenating them to the noisy latent. We use the TCMonoDepth [Li et al., 2021] model to estimate depth maps in the RGB space and rescale them into the size of the latent codes. When generating the kth edited frame, we concatenate its depth map $\boldsymbol{m}^k$, the latent of the previous edited frame $\hat{\boldsymbol{z}}_t^{k-1}$, the previous depth map $\boldsymbol{m}^{k-1}$, to the noisy latent $\hat{\boldsymbol{z}}_t$. Instead, the semantic conditions are used as the input of the cross-attention modules. We design a lightweight adaptor to combine the CLIP's embedding and the VAE latent of the reference frame so that the injected semantics contains both class-wise and patch-wise information.

3.3 MODEL DESIGN OF PROPDPM

The main challenges of video editing are ensuring temporal consistency across all frames and maintaining per-frame quality. PropDPM addresses the first challenge by editing a video in an auto-regressive manner, conditioning on the true depth sequence to ensure temporal coherence across frames. However, due to the intrinsic error accumulation issue of auto-regressive modeling, the image quality of the edited frames degrades as the frame index increases. While the early edited frames contain rich details, the later edited ones become smooth and suffer from color shifting.

To alleviate the error accumulation issue, we propose two complementary solutions. First, we design an appearance adaptor that merges the class-level and patch-wise information of the reference frame. The output of this adaptor is sent to cross-attention modules. During inference, we use a fixed reference frame for each video when auto-regressively synthesizing frames. A fixed reference frame serves as an anchor to ameliorate the degradation. Second, we apply the Zero-Terminal-SNR [Lin et al., 2023] technique to train the diffusion model, which bridges the gap between the starting noise's strength during inference and the largest noise level during training. This technique improves the image quality of the generated frame in each iteration.

3.3.1 APPEARANCE ADAPTOR

We design a lightweight adaptor to fuse the class-level and pixel-level features of the reference frame. The adaptor preserves the spatial correspondence between the fused tokens and the reference image. In detail, we first use the VAE to extract the latent of the reference image, $\boldsymbol{z}^{\#} \in \mathbb{R}^{4\times h\times w}$. The latent codes of VAE have good spatial correspondence to the original images. We use a nonlinear network to decrease the redundant spatial resolution of latent $\boldsymbol{z}^{\#}$ by a factor of $\times 2$ but increase the channel dimension to preserve more information. The resultant feature is in size of $\mathbb{R}^{l/2\times h/2\times w/2}$,

where l is the length of each CLIP embedding. On the other hand, we use the CLIP model to extract the semantics of the reference image. We have a global class token $\tau(\boldsymbol{x}^{\#})_{\mathrm{c}} \in \mathbb{R}^{l\times 1}$ and patch-wise tokens $\tau(\boldsymbol{x}^{\#})_{\mathrm{p}} \in \mathbb{R}^{l\times h'\times w'}$. We utilize another nonlinear network to downsample the token dimension and adjust their spatial resolution to $\mathbb{R}^{l/2\times h/2\times w/2}$. Finally, we apply the third nonlinear module to merge the transformed CLIP's and the VAE's features into a fused feature in size of $\mathbb{R}^{l\times h/2\times w/2}$. We concatenate it with the untouched class-level token and use it (reshaped into the size of $\mathbb{R}^{l\times(hw/4+1)}$) as the input of cross-attention modules. Since the fused tokens contain rich global and local information, PropDPM can generate a target frame that better matches the reference's appearance.

3.3.2 Zero-Terminal-SNR Noise Schedule

Diffusion models are trained to estimate the noise in the noisy intermediate state $\boldsymbol{x}_t$ for $t \in [1, \ldots, T]$, where $\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_0 + \sqrt{1-\alpha_t^2}\epsilon$. In the vanilla DDPM, the noise schedule is set to be $1 = \alpha_0 > \alpha_1 > \cdots > \alpha_T > 0$, where the terminal signal-to-noise-ratio (SNR), $SNR(t) = \alpha_t^2/(1-\alpha_t^2)$, is greater than 0. This means the strongest noise, that the obtained DDPM can handle, is $\boldsymbol{x}_T = \alpha_T \boldsymbol{x}_0 + \sqrt{1-\alpha_T^2}\epsilon$ rather than the pure noise ϵ. However, during inference, most samplers start from pure noise. This gap may incur the degradation of the generated data. To fix this issue, Lin *et al.* [ ] propose a novel noise schedule, termed Zero-Terminal-SNR, which forces the SNR(T) to be zero and make the UNet $\boldsymbol{v}_\theta(\boldsymbol{z}_t)$ to predict the $\boldsymbol{v}$-value instead of noise ϵ. The v-value is defined as $\boldsymbol{v}_t = \alpha_t \epsilon - \sqrt{(1-\alpha_t^2)}\boldsymbol{x}_0$. We follow the Zero-Terminal-SNR strategy for training our PropDPM model. The experimental results verify the effectiveness of alleviating the color-shifting issue.

3.3.3 Training

The PropDPM is initialized from the Stable-Diffusion-v1.5. We train the PropDPM model on the combination of a public video dataset, WebVid-10M [Bain et al., 2021], and a self-collected private dataset. For the public one, we randomly sample 1 million videos, while the self-collected dataset contains 20 thousand high-resolution videos without watermarks. From each video, we sample at most 30 frames with a step size of *four*. These frames are then center-cropped into squares and resized into the shape of 256×256. During training, we randomly select three frames from a video to form a triplet: the reference frame, the previous frame, and the target frame.

4 Application

MagicProp can edit any arbitrary region in the given video. In Figure 4 and Figure 5, we show the rendered videos. We use masks and ControlNet to localize and modify certain parts. The masks can be either provided by users or extracted by a segmentation model (*e.g.*, Segment-Anything).

Through extensive experiments, we find MagicProp can robustly edit videos up to 30 frames. Degradation, such as over-smoothing and artifacts, may appear when the length of videos exceeds 30 frames due to the intrinsic error accumulation of Auto-regressive inference. For future work, we aim to improve the current MagicProp framework for processing longer videos.

References

M. Bain, A. Nagrani, G. Varol, and A. Zisserman. Frozen in time: A joint video and image encoder for end-to-end retrieval. In *IEEE International Conference on Computer Vision*, 2021.

O. Bar-Tal, D. Ofri-Amar, R. Fridman, Y. Kasten, and T. Dekel. Text2LIVE: Text-Driven Layered Image and Video Editing, May 2022. arXiv:2204.02491 [cs].

A. Blattmann, R. Rombach, H. Ling, T. Dockhorn, S. W. Kim, S. Fidler, and K. Kreis. Align your Latents: High-Resolution Video Synthesis with Latent Diffusion Models, Apr. 2023. arXiv:2304.08818 [cs].

D. Ceylan, C.-H. P. Huang, and N. J. Mitra. Pix2Video: Video Editing using Image Diffusion, Mar. 2023. arXiv:2303.12688 [cs].

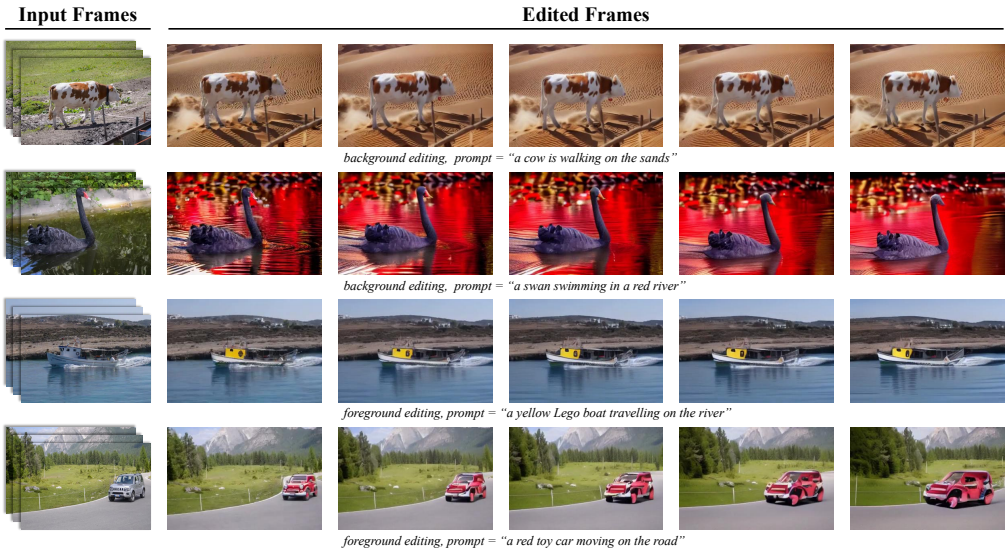

Figure 4: Examples for local editing—background (the top two) and foreground editing (the bottom two).

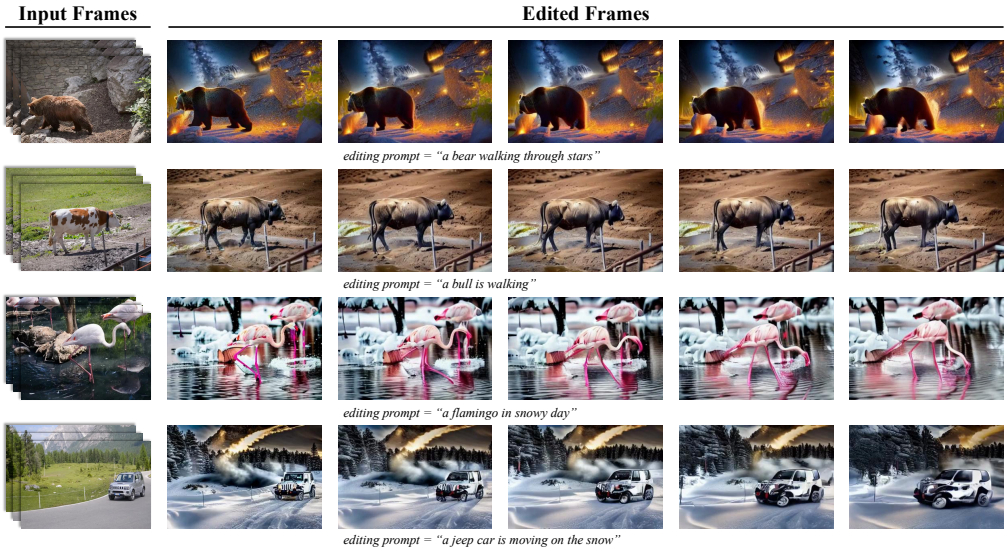

Figure 5: Examples for global editing.

W. Chen, J. Wu, P. Xie, H. Wu, J. Li, X. Xia, X. Xiao, and L. Lin. Control-A-Video: Controllable Text-to-Video Generation with Diffusion Models, May 2023. arXiv:2305.13840 [cs].

P. Couairon, C. Rambour, J.-E. Haugeard, and N. Thome. VidEdit: Zero-Shot and Spatially Aware Text-Driven Video Editing, June 2023. arXiv:2306.08707 [cs].

P. Esser, J. Chiu, P. Atighehchian, J. Granskog, and A. Germanidis. Structure and Content-Guided Video Synthesis with Diffusion Models, Feb. 2023. arXiv:2302.03011 [cs].

M. Geyer, O. Bar-Tal, S. Bagon, and T. Dekel. TokenFlow: Consistent Diffusion Features for Consistent Video Editing, July 2023. arXiv:2307.10373 [cs].

A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch, and D. Cohen-Or. Prompt-to-Prompt Image Editing with Cross Attention Control, Aug. 2022. arXiv:2208.01626 [cs].

J. Ho, A. Jain, and P. Abbeel. Denoising Diffusion Probabilistic Models, Dec. 2020. arXiv:2006.11239 [cs, stat].

J. Ho, T. Salimans, A. Gritsenko, W. Chan, M. Norouzi, and D. J. Fleet. Video Diffusion Models. Technical Report arXiv:2204.03458, arXiv, June 2022. arXiv:2204.03458 [cs] type: article.

J. Huang, L. Sigal, K. M. Yi, O. Wang, and J.-Y. Lee. INVE: Interactive Neural Video Editing, July 2023a. arXiv:2307.07663 [cs].

L. Huang, D. Chen, Y. Liu, Y. Shen, D. Zhao, and J. Zhou. Composer: Creative and Controllable Image Synthesis with Composable Conditions, Feb. 2023b. arXiv:2302.09778 [cs].

J. Karras, A. Holynski, T.-C. Wang, and I. Kemelmacher-Shlizerman. DreamPose: Fashion Image-to-Video Synthesis via Stable Diffusion, May 2023. arXiv:2304.06025 [cs].

Y. Kasten, D. Ofri, O. Wang, and T. Dekel. Layered Neural Atlases for Consistent Video Editing, Sept. 2021. arXiv:2109.11418 [cs].

L. Khachatryan, A. Movsisyan, V. Tadevosyan, R. Henschel, Z. Wang, S. Navasardyan, and H. Shi. Text2Video-Zero: Text-to-Image Diffusion Models are Zero-Shot Video Generators, Mar. 2023. arXiv:2303.13439 [cs].

S. Li, Y. Luo, Y. Zhu, X. Zhao, Y. Li, and Y. Shan. Enforcing temporal consistency in video depth estimation. In *2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 1145–1154, 2021. doi: 10.1109/ICCVW54120.2021.00134.

J. H. Liew, H. Yan, D. Zhou, and J. Feng. MagicMix: Semantic Mixing with Diffusion Models, Oct. 2022. arXiv:2210.16056 [cs].

S. Lin, B. Liu, J. Li, and X. Yang. Common Diffusion Noise Schedules and Sample Steps are Flawed, May 2023. arXiv:2305.08891 [cs].

C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, and S. Ermon. SDEdit: Guided Image Synthesis and Editing with Stochastic Differential Equations. Technical Report arXiv:2108.01073, arXiv, Jan. 2022. arXiv:2108.01073 [cs] type: article.

E. Molad, E. Horwitz, D. Valevski, A. R. Acha, Y. Matias, Y. Pritch, Y. Leviathan, and Y. Hoshen. Dreamix: Video Diffusion Models are General Video Editors, Feb. 2023. arXiv:2302.01329 [cs].

H. Ni, C. Shi, K. Li, S. X. Huang, and M. R. Min. Conditional Image-to-Video Generation with Latent Flow Diffusion Models, Mar. 2023. arXiv:2303.13744 [cs].

C. Qi, X. Cun, Y. Zhang, C. Lei, X. Wang, Y. Shan, and Q. Chen. FateZero: Fusing Attentions for Zero-shot Text-based Video Editing, Mar. 2023. arXiv:2303.09535 [cs].

R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-Resolution Image Synthesis with Latent Diffusion Models, Apr. 2022. arXiv:2112.10752 [cs].

J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli. Deep Unsupervised Learning using Nonequilibrium Thermodynamics. page 10, 2015.

X. Wang, H. Yuan, S. Zhang, D. Chen, J. Wang, Y. Zhang, Y. Shen, D. Zhao, and J. Zhou. VideoComposer: Compositional Video Synthesis with Motion Controllability, June 2023. arXiv:2306.02018 [cs].

J. Z. Wu, Y. Ge, X. Wang, W. Lei, Y. Gu, Y. Shi, W. Hsu, Y. Shan, X. Qie, and M. Z. Shou. Tune-A-Video: One-Shot Tuning of Image Diffusion Models for Text-to-Video Generation, Mar. 2023. arXiv:2212.11565 [cs].

S. Yang, Y. Zhou, Z. Liu, and C. C. Loy. Rerender A Video: Zero-Shot Text-Guided Video-to-Video Translation, June 2023. arXiv:2306.07954 [cs].

L. Zhang and M. Agrawala. Adding Conditional Control to Text-to-Image Diffusion Models, Feb. 2023. arXiv:2302.05543 [cs].

M. Zhao, R. Wang, F. Bao, C. Li, and J. Zhu. ControlVideo: Adding Conditional Control for One Shot Text-to-Video Editing, May 2023. arXiv:2305.17098 [cs].

D. Zhou, W. Wang, H. Yan, W. Lv, Y. Zhu, and J. Feng. MagicVideo: Efficient Video Generation With Latent Diffusion Models, May 2023. arXiv:2211.11018 [cs].

A SUPPLEMENTARY

Implementation of the adaptor that fuses the class-level and pixel-level information of the edited reference frame.

```
import math
import numpy as np
import torch
import torch.nn as nn
from einops import rearrange

class Embedding_Adapter(nn.Module):
    """
        Fusing the CLIP embeddings and VAE latents of images.
    """
    def __init__(self, ca_emb_size=768):
        super(Embedding_Adapter, self).__init__()
        assert ca_emb_size % 2 == 0
        self.clip_downsample_emb = nn.Sequential(
                        nn.Conv2d(ca_emb_size, ca_emb_size//2,
                            kernel_size=1, stride=1,
                            padding=0, bias=True),
                        nn.SiLU(),
                        nn.Conv2d(ca_emb_size//2, ca_emb_size//2,
                            kernel_size=1, stride=1,
                            padding=0, bias=True),
                        nn.SiLU(),
                        )
        self.vae_upsample_chn_down_spatial = nn.Sequential(
                        nn.Conv2d(4, ca_emb_size//2,
                            kernel_size=3, stride=1,
                            padding=1, bias=True),
                        nn.SiLU(),
                        nn.MaxPool2d(2),
                        nn.Conv2d(ca_emb_size//2, ca_emb_size//2,
                            kernel_size=1, stride=1,
                            padding=0, bias=True),
                        nn.SiLU(),
                        )
        self.mix_clip_vae = nn.Sequential(
                        nn.Conv2d(ca_emb_size, ca_emb_size,
                            kernel_size=3, stride=1,
                            padding=1, bias=True),
                        nn.SiLU(),
                        nn.Conv2d(ca_emb_size, ca_emb_size,
                            kernel_size=3, stride=1,
                            padding=1, bias=True),
                        )

    def forward(self, clip, vae):
        bs_vae, _, h_vae, w_vae = vae.size()
        bs_emb, hw, _ = clip.size()
        h_emb = int(math.sqrt(hw-1))
        assert (bs_vae==bs_emb) and ((hw-1)%h_emb==0)

        clip_cls = clip[:, 0:1, :]
```

```
        ## adjusting clip_patch embeddings,
        clip_patch = clip[:, 1:, :]
        clip_patch = rearrange(clip_patch, 'b (h w) c -> b c h w',
                               h=h_emb, w=h_emb)
        clip_patch = torch.nn.functional.interpolate(clip_patch,
                               size=(h_vae//2, w_vae//2),
                               mode="bilinear")
        # (b, h_emb^2, 768) -> (b, 384, h//2, w//2)
        clip_patch = self.clip_downsample_emb(clip_patch)

        ## adjusting vae latents
        # (b, 4, h, w) -> (b, 384, h//2, w//2)
        vae = self.vae_upsample_chn_down_spatial(vae)

        ## fusing, #(b, 1+hw//4, 768)
        clip_vae = torch.cat([clip_patch, vae], dim=1)
        clip_vae = self.mix_clip_vae(clip_vae)
        clip_vae = rearrange(clip_vae, 'b c h w -> b (h w) c')
        return torch.cat([clip_cls, clip_vae], dim=1)
```

Listing 1: Appearance adaptor